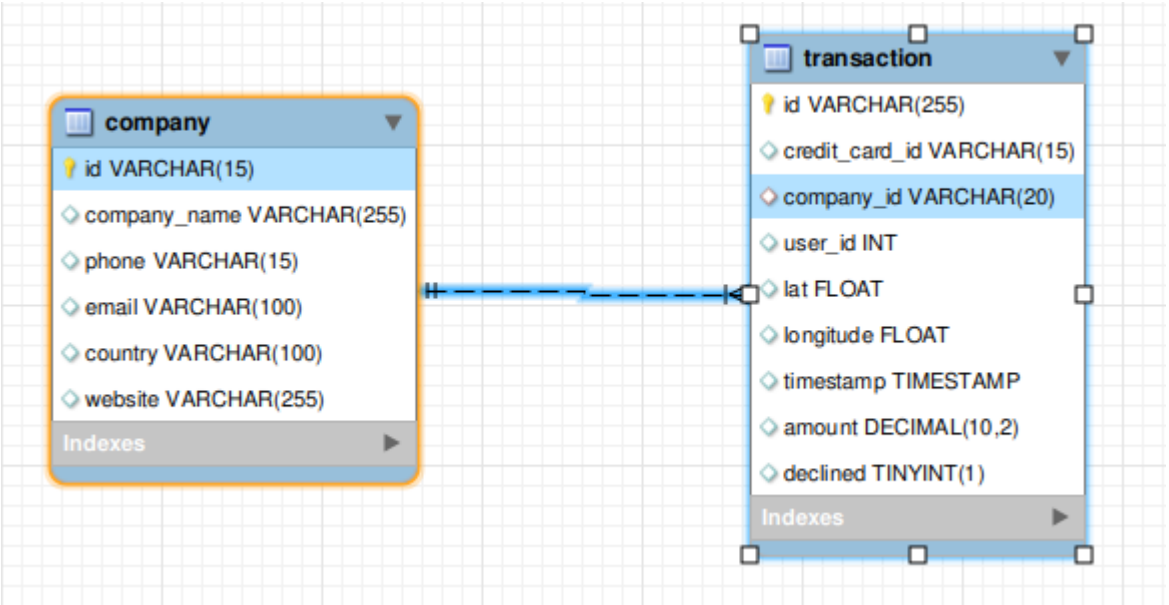
# Tasca Sprint 2

## 1. Nivell 1

**Exercici 1**
**A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.**




La tabla 'company' tiene 6 campos y nos da información detallada de las empresas: el nombre de la empresa, teléfono, email, país y web. La PK es 'id'.

La tabla 'transaction' tiene 9 campos y nos da información respecto a las transacciones realizadas por las empresas, es decir, las ventas que hicieron. Encontramos los campos:

- id (PK)
- credit_card_id: identificador de la tarjeta de crédito desde la cual se hizo la transferéncia
- company_id (FK): identificador de la empresa
- user_id: identificador del usuario hizo la transacción
- lat y longitude: las coordenadas desde donde se hizo la transacción o compra
- timestamp: la fecha exacta en la que se hizo
- amount: cantidad de dinero pagada
- declined: indica si la transacción fue aceptada o no

La tabla ‘transaction’ tiene elevada granularidad pues tiene 9 campos para detallar las transacciones, mientras que la tabla ‘company’ tiene menor granularidad ya que solo tiene 6 campos.

La cardinalidad entre tablas es 1:N, es decir, cada empresa de la tabla ‘company’ puede aparecer más de una vez en la tabla ‘transaction’, ya que las empresas pueden hacer varias transacciones.

Finalmente, la tabla transaction representa la tabla de hechos, la cual se vincula con la tabla de dimensiones (company) mediante la FK ‘company_id’.

---

**Exercici 2**
**Utilitzant JOIN realitzaràs les següents consultes:**

**Llistat dels països que estan generant vendes.**

```
SELECT DISTINCT c.country
FROM `transaction` AS t
JOIN company AS c
ON t.company_id = c.id
WHERE declined = 0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | country |
|---|---|
| 1 | Netherlands |
| 2 | Sweden |
| 3 | Ireland |
| 4 | United States |
| 5 | Belgium |
| 6 | Canada |
| 7 | Germany |
| 8 | Norway |
| 9 | France |
| 10 | Italy |
| 11 | United Kingdom |
| 12 | New Zealand |
| 13 | China |
| 14 | Spain |
| 15 | Australia |

Result 21

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 18:54:09 | SELECT DISTINCT c.country FROM `transaction` AS t JOIN company AS c ON t.company_id = c.id WHERE declined = 0 | 16 row(s) returned |

Hago una join entre las dos tablas con el filtro WHERE con declined =0 para ver sólo las ventas realizadas. En el select uso DISTINC para obtener países diferentes.

**Des de quants països es generen les vendes.**

```
14 •  SELECT count(DISTINCT c.country) AS total_paises
15    FROM `transaction` AS t
16    JOIN company AS c
17    ON t.company_id = c.id
18    WHERE declined = 0;
19
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | total_paises |
|---|---|
| 1 | 15 |

Result 22

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 18:55:04 | SELECT count(DISTINCT c.country) AS total_paises FROM `transaction` AS t JOIN company AS c ON t.company_id = c.id WHERE declined ... | 1 row(s) returned |

La misma query que antes pero ahora añado el count para contar los países que generan ventas.

---

**Identifica la companyia amb la mitjana més gran de vendes.**

```
24 •  SELECT c.id, company_name, round(avg(amount),2) as avg_vendes
25    FROM `transaction` AS t
26    JOIN company AS c
27    ON t.company_id = c.id
28    WHERE declined = 0
29    GROUP BY c.id, company_name
30    ORDER BY avg_vendes DESC
31    LIMIT 1;
32
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

| # | id | company_name | avg_vendes |
|---|---|---|---|
| 1 | b-2222 | Ac Fermentum Incorporated | 284.91 |

Result 6

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 17:54:48 | SELECT c.id, company_name, round(avg(amount),2) as avg_vendes FROM `transaction` AS t JOIN company AS c ON t.company_id = c.id ... | 1 row(s) returned |

Hago una join, luego agrupo por empresa, calculo la media redondeando a 2 decimales, ordeno por la media de mayor a menor de acuerdo al campo 'avg_vendes' sin considerar los decimales. Finalmente me quedo con el mayor al aplicar LIMIT 1.

## Exercici 3
## Utilitzant només subconsultes (sense utilitzar JOIN):

### Mostra totes les transaccions realitzades per empreses d'Alemanya.

```
SELECT t.id AS transaccion, t.company_id AS empresa
FROM `transaction` AS t
WHERE declined = 0
AND
EXISTS (
    SELECT id FROM company AS c
    WHERE t.company_id = c.id AND
    country = 'Germany'
    );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

| # | transaccion | empresa |
|---|---|---|
| 1 | 00138D3B-206D-4C03-94B7-63A2... | b-2222 |
| 2 | 0013C1B6-3B84-4D6C-8154-E2B... | b-2222 |
| 3 | 00201A11-2E62-44C4-941D-198F... | b-2222 |
| 4 | 00235618-0A5C-4D49-9DCB-B3A... | b-2222 |
| 5 | 005A5A7B-1F1A-4B6C-9B15-162... | b-2222 |
| 6 | 00687139-48B2-4FFA-8E73-B203... | b-2222 |
| 7 | 0074F4DD-32F1-4827-8758-5589... | b-2222 |
| 8 | 00AAB9CD-39D6-4DCB-8A1D-13... | b-2222 |
| 9 | 00BE09D4-6920-47D8-ABE8-325... | b-2222 |
| 10 | 00DA0383-E048-4577-8ED1-3C56... | b-2222 |
| 11 | 00DD11DE-ED01-4BBD-93A0-17... | b-2222 |

transaction 19

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 18:53:14 | SELECT t.id AS transaccion, t.company_id AS empresa FROM `transaction` AS t WHERE declined = 0 AND EXISTS ( SELECT id FROM co... | 13269 row(s) returned |

Hago una subconsulta con EXISTS de manera que me devuelve las empreses cuyo id de company hace match con el company_id de 'transactions' estableciendo el filtro por el país. También con declined = 0 para obtener las transacciones aceptadas.

---

### Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

```
SELECT c.id, c.company_name
FROM `company` AS c
WHERE EXISTS(
            SELECT t.id FROM `transaction` AS t
            WHERE t.company_id = c.id
            AND t.amount >  (
                            SELECT AVG(amount)
                            FROM `transaction` WHERE declined=0)
AND declined = 0);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| # | id | company_name |
|---|---|---|
| 1 | b-2222 | Ac Fermentum Incorporated |
| 2 | b-2226 | Magna A Neque Industries |
| 3 | b-2230 | Fusce Corp. |
| 4 | b-2234 | Convallis In Incorporated |
| 5 | b-2238 | Ante Iaculis Nec Foundation |
| 6 | b-2242 | Donec Ltd |
| 7 | b-2246 | Sed Nunc Ltd |
| 8 | b-2250 | Amet Nulla Donec Corpor... |
| 9 | b-2254 | Nascetur Ridiculus Mus Inc. |
| 10 | b-2258 | Vestibulum Lorem PC |
| 11 | b-2262 | Gravida Sagittis LLP |
| 12 | b-2266 | Mus Aenean Eget Founda... |
| 13 | b-2270 | Dis Parturient Institute |
| 14 | b-2274 | Sed LLC |
| 15 | b-2278 | Arcu LLP |
| 16 | b-2282 | Pretium Neque Corp. |

company 17 | Apply | Revert

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 18:21:50 | SELECT c.id, c.company_name FROM `company` AS c WHERE EXISTS( SELECT t.id FROM `transaction` AS t WHERE t.company_id = c.id AND t.amount > ( ... | 100 row(s) |

Hago una subquery donde se muestran las empresas cuyo amount es mayor a la media de amount de todas las transacciones. Aplico el filtro WHERE tanto en la subquery como fuera para que me considere solo las transacciones aceptadas.

---

**Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.**

```
54 • SELECT id, company_name
55   FROM company AS c
56   WHERE NOT EXISTS
57       (SELECT company_id
58       FROM `transaction` AS t
59       WHERE c.id = t.company_id
60       AND declined= 0);
61
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| # | id | company_name |
|---|---|---|
| * | NULL | NULL |

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 19:11:48 | SELECT id, company_name FROM company AS c WHERE NOT EXISTS (SELECT company_id FROM `transaction` AS t WHERE c.id ... | 0 row(s) returned |

Hago una subquery con NOT EXISTS de manera que se muestran aquellas empresas que no tienen transacciones porque el id de company no hace match con el company_id de transaction.

---

**Exercici 4**

**La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.**

```
75 • USE transactions;
76
77 • CREATE TABLE IF NOT EXISTS credit_card(
78       id VARCHAR (15) PRIMARY KEY,
79       iban VARCHAR (34) NOT NULL,
80       pan VARCHAR(30),
81       pin CHAR(4),
82       cvv CHAR(3),
83       expiring_date DATE
84   );
85
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 18:45:35 | CREATE TABLE IF NOT EXISTS credit_card( id VARCHAR (15) PRIMARY KEY, iban VARCHAR (34) NOT NULL, pan ... | 0 row(s) affected |

En importar los datos con los tipos de datos que se muestra en la imagen, me da error en la columna ‘expiring_date’ por lo que voy a modificar esta columna a VARCHAR (ver en las 2 siguientes imágenes).

```
2 • ALUES ('CcU-2938', 'TR3019503122135768176386 61', '5424465566813633', '3257', '984', '10/30/22');
3 • ALUES ('CcU-2945', 'DO2685476374853747521656 8689', '5142423821948828', '9080', '887', '08/24/23');
4 • ALUES ('CcU-2952', 'BG45IVQL52710525608255', '4556 453 55 5287', '4598', '438', '06/29/21');
5 • ALUES ('CcU-2959', 'CR7242477244335841535', '372461377349375', '3583', '667', '02/24/23');
6 • ALUES ('CcU-2966', 'BG72LKTQ15627628377363', '448566 886747 7265', '4900', '130', '10/29/24');
7 • ALUES ('CcU-2973', 'PT8780622813509242945 6346', '544 58654 54343 384', '8760', '887', '01/30/25');
8 • ALUES ('CcU-2980', 'DE39241881883086277136', '402400 7145845969', '5075', '596', '07/24/22');
9 • ALUES ('CcU-2987', 'GE8968143483774878181 3', '3763 747687 76666', '2298', '797', '10/31/23');
10 • ALUES ('CcU-2994', 'BH6271442836806676529 4', '344283273252593', '7545', '595', '02/28/22');
11 • ALUES ('CcU-3001', 'CY4908742665477458126683 2110', '511722 924833 2244', '9562', '867', '09/16/22');
12 • ALUES ('CcU-3008', 'LU5072166936161192 30', '4485744464433884', '1856', '740', '04/05/25');
13 • ALUES ('CcU-3015', 'PS11939821629571596834245 6821', '3784 662233 17389', '3246', '822', '01/31/22');
14 • ALUES ('CcU-3022', 'GT91695162850556977423121857', '5164 1379 4842 3951', '5610', '342', '04/25/25');
15 • ALUES ('CcU-3029', 'AZ6231741398244141812373974 6', '3429 279566 77631', '9708', '505', '09/02/23');
16 • ALUES ('CcU-3036', 'AZ3933600292584286584394 1994', '3768 451556 48766', '2232', '565', '10/27/25');
17 • ALUES ('CcU-3043', 'TN6488143310514852179535', '455676 6437463635', '5969', '196', '06/07/25');
18 • ALUES ('CcU-3050', 'FR5167744369175836831854477', '4024007123722', '4834', '126', '10/09/23');
19 • ALUES ('CcU-3057', 'LU931822574697545215', '3484 621767 21237', '6805', '848', '09/14/25');
20 • ALUES ('CcU-3064', 'PS14696554544925337762727 3133', '3467 732741 26810', '3865', '498', '06/03/25');
21 • ALUES ('CcU-3071', 'NO8923814763512', '3464 789562 23352', '6625', '661', '12/20/23');
22 • ALUES ('CcU-3078', 'IS0251271458846232795487 33', '4539 322 74 2377', '9405', '720', '03/08/23');
23 • ALUES ('CcU-3085', 'BE63114723972437', '5266 3346 1135 1687', '7241', '413', '05/10/23');
24 • ALUES ('CcU-3092', 'RO65LSOD1166122125447487', '3488 754223 46253', '9417', '594', '12/19/22');
25 • ALUES ('CcU-3099', 'PT2610527535682370553721 8', '448 55418 98863 789', '5612', '564', '01/22/23');
26 • ALUES ('CcU-3106', 'AT684251637751136592', '349547146395283', '9733', '209', '01/27/24');
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 18:53:26 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR30195031221357... | Error Code: 1292. Incorrect date value: '10/30/22' for column 'expiring_date' at row 1 |

```
87 • ALTER TABLE credit_card
88   MODIFY COLUMN expiring_date VARCHAR(15);
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 18:57:41 | ALTER TABLE credit_card MODIFY COLUMN expiring_date VARCHAR(15) | 0 row(s) affected<br>Records: 0 Duplicates: 0 Warnings: 0 |

Una vez modificada la columna a VARCHAR (15), si pude importar los datos, como se muestra a continuación:

```
2 • ALUES ('CcU-2938', 'TR3019503122135768176386 61', '5424465566813633', '3257', '984', '10/30/22');
3 • ALUES ('CcU-2945', 'DO2685476374853747521656 8689', '5142423821948828', '9080', '887', '08/24/23');
4 • ALUES ('CcU-2952', 'BG45IVQL52710525608255', '4556 453 55 5287', '4598', '438', '06/29/21');
5 • ALUES ('CcU-2959', 'CR7242477244335841535', '372461377349375', '3583', '667', '02/24/23');
6 • ALUES ('CcU-2966', 'BG72LKTQ15627628377363', '448566 886747 7265', '4900', '130', '10/29/24');
7 • ALUES ('CcU-2973', 'PT8780622813509242945 6346', '544 58654 54343 384', '8760', '887', '01/30/25');
8 • ALUES ('CcU-2980', 'DE39241881883086277136', '402400 7145845969', '5075', '596', '07/24/22');
9 • ALUES ('CcU-2987', 'GE8968143483774878181 3', '3763 747687 76666', '2298', '797', '10/31/23');
10 • ALUES ('CcU-2994', 'BH6271442836806676529 4', '344283273252593', '7545', '595', '02/28/22');
11 • ALUES ('CcU-3001', 'CY4908742665477458126683 2110', '511722 924833 2244', '9562', '867', '09/16/22');
12 • ALUES ('CcU-3008', 'LU5072166936161192 30', '4485744464433884', '1856', '740', '04/05/25');
13 • ALUES ('CcU-3015', 'PS11939821629571596834245 6821', '3784 662233 17389', '3246', '822', '01/31/22');
14 • ALUES ('CcU-3022', 'GT91695162850556977423121857', '5164 1379 4842 3951', '5610', '342', '04/25/25');
15 • ALUES ('CcU-3029', 'AZ6231741398244141812373974 6', '3429 279566 77631', '9708', '505', '09/02/23');
16 • ALUES ('CcU-3036', 'AZ3933600292584286584394 1994', '3768 451556 48766', '2232', '565', '10/27/25');
17 • ALUES ('CcU-3043', 'TN6488143310514852179535', '455676 6437463635', '5969', '196', '06/07/25');
18 • ALUES ('CcU-3050', 'FR5167744369175836831854477', '4024007123722', '4834', '126', '10/09/23');
19 • ALUES ('CcU-3057', 'LU931822574697545215', '3484 621767 21237', '6805', '848', '09/14/25');
20 • ALUES ('CcU-3064', 'PS14696554544925337762727 3133', '3467 732741 26810', '3865', '498', '06/03/25');
21 • ALUES ('CcU-3071', 'NO8923814763512', '3464 789562 23352', '6625', '661', '12/20/23');
22 • ALUES ('CcU-3078', 'IS0251271458846232795487 33', '4539 322 74 2377', '9405', '720', '03/08/23');
23 • ALUES ('CcU-3085', 'BE63114723972437', '5266 3346 1135 1687', '7241', '413', '05/10/23');
24 • ALUES ('CcU-3092', 'RO65LSOD1166122125447487', '3488 754223 46253', '9417', '594', '12/19/22');
25 • ALUES ('CcU-3099', 'PT2610527535682370553721 8', '448 55418 98863 789', '5612', '564', '01/22/23');
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 4996 | 18:59:39 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9577', 'XX15891407859480... | 1 row(s) affected |
| 4997 | 18:59:39 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9578', 'XX99153964645611... | 1 row(s) affected |
| 4998 | 18:59:39 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9579', 'XX29639309158717... | 1 row(s) affected |
| 4999 | 18:59:39 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX78125888985195... | 1 row(s) affected |
| 5000 | 18:59:39 | INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX91567051640538... | 1 row(s) affected |

Ahora, para poder modificar de nuevo los datos a tipo DATE, primero modifiqué el formato mediante UPDATE / SET y la función STR_TO_DATE. Finalmente, ya pude modificar el tipo de dato con ALTER TABLE / MODIFY COLUMN.

```
91 •  UPDATE credit_card
92    SET expiring_date = STR_TO_DATE (expiring_date, '%m/%d/%y')
93    LIMIT 6000;
94
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 19:05:03 | UPDATE credit_card SET expiring_date = STR_TO_DATE (expiring_date, '%m/%d/%y') LIMIT 6000 | 5000 row(s) affected<br>Rows matched: 5000 Changed: 5000 Warnings: 0 |

```
95 •  ALTER TABLE credit_card
96    MODIFY COLUMN expiring_date DATE;
97
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 19:05:44 | ALTER TABLE credit_card MODIFY COLUMN expiring_date DATE | 5000 row(s) affected<br>Records: 5000 Duplicates: 0 Warnings: 0 |

Ahora ya tenemos los datos correctamente cargados y con el tipo de dato de las variables bien establecidos.

Intento poner la FK en ‘transactions’ para crear la relación con la tabla nueva pero no me deja, debe haber algún dato que no coincide en ambas tablas, es decir, alguna PK que no se relaciona.

```
100 •  ALTER TABLE `transaction`
101    ADD CONSTRAINT fk_transaction_credit_card
102    FOREIGN KEY (credit_card_id)
103    REFERENCES credit_card(id);
```

Action Output

| # | Time | Action |
|---|---|---|
| 1 | 19:20:44 | ALTER TABLE `transaction` ADD CONSTRAINT fk_transaction_credit_card FOREIGN KEY (credit_card_id) REFERENCES credit_card... |

Hacemos una LEFT JOIN desde ‘transaction’ para comprobar si hay alguna tarjeta de crédito que aparece en ‘transaction’ pero no en la tabla ‘credit_card’. Al hacer la query, vemos que efectivamente hay una tarjeta que no aparece en ‘credit_card’ (ver en la siguiente imagen), por lo que vamos a proceder a añadirla a esta tabla.

```
105 •  SELECT DISTINCT t.credit_card_id
106    FROM `transaction` AS t
107    LEFT JOIN credit_card AS cc
108    ON t.credit_card_id = cc.id
109    WHERE cc.id IS NULL;
110
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | credit_card_id |
|---|---|
| 1 | CcU-9999 |

Result 22

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 19:26:51 | SELECT DISTINCT t.credit_card_id FROM `transaction` AS t LEFT JOIN credit_card AS cc ON t.credit_card_id = cc.id WHERE cc.id IS ... | 1 row(s) returned |

Intenté añadir sólo el id en la tabla ‘credit_card’ pero no me dejó porque el campo ‘iban’ es NOT NULL, es decir, obligatoriamente tiene que estar. Para solucionar este contratiempo, se me ocurrió rellenar con ‘temporary_need_update’, que viene a ser como ‘temporal, necesita actualizarse’. Me pareció la mejor forma para dar a entender a alguien que abre la tabla por primera vez y se encuentra con esto, que sepa automáticamente lo que ocurre.

```
112 •  INSERT INTO credit_card
113    (id, iban) VALUES ('CcU-9999', 'temporary__need_update');
```

Action Output

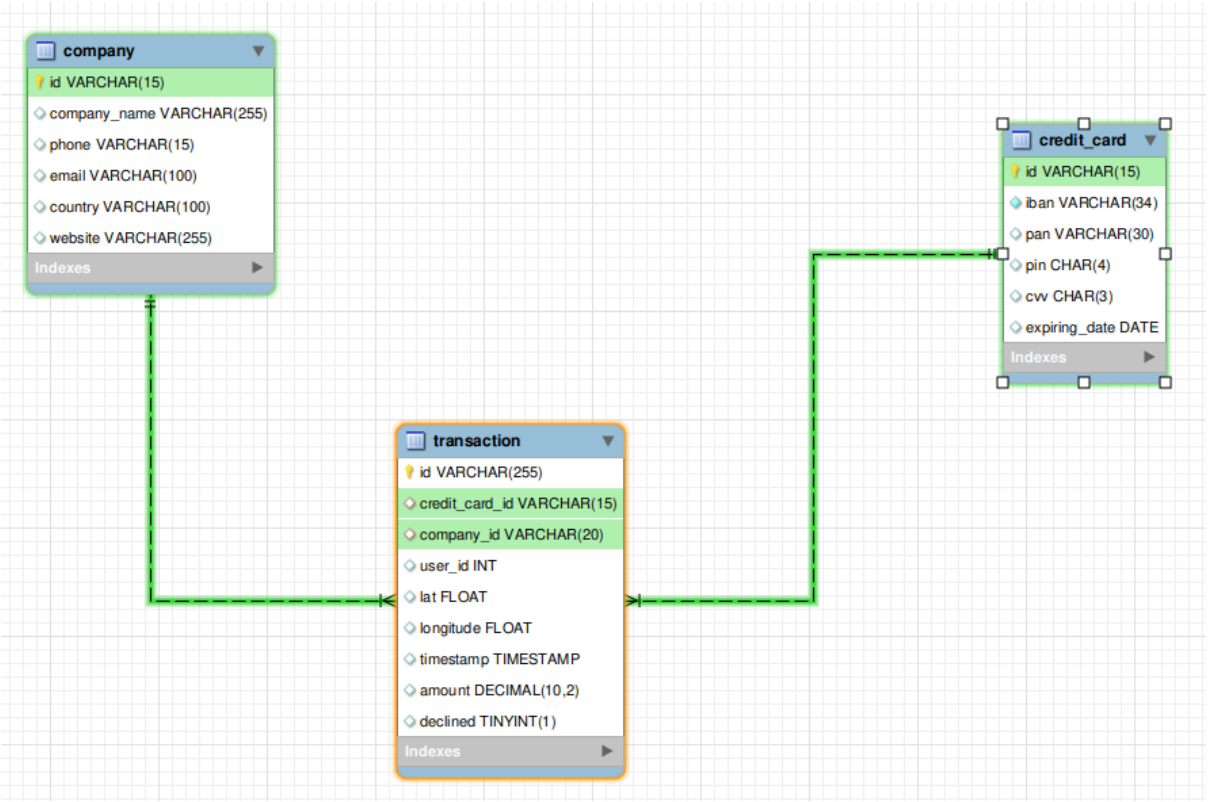
| # | Time | Action | Message |
|---|---|---|---|
| 1 | 19:53:06 | INSERT INTO credit_card (id, iban) VALUES ('CcU-9999', 'temporary__need_update') | 1 row(s) affected |

Finalmente, establecemos la relación entre las tablas mediante la configuración de la FK.

```
115
116 •  ALTER TABLE `transaction`
117    ADD CONSTRAINT fk_transaction_credit_card
118    FOREIGN KEY (credit_card_id)
119    REFERENCES credit_card(id);
120
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 19:58:43 | ALTER TABLE `transaction` ADD CONSTRAINT fk_transaction_credit_card FOREIGN KEY (credit_card_id) REFERENCES credit_card... | 100001 row(s) affected Records: 100001 Duplicates: 0 Warnings: 0 |

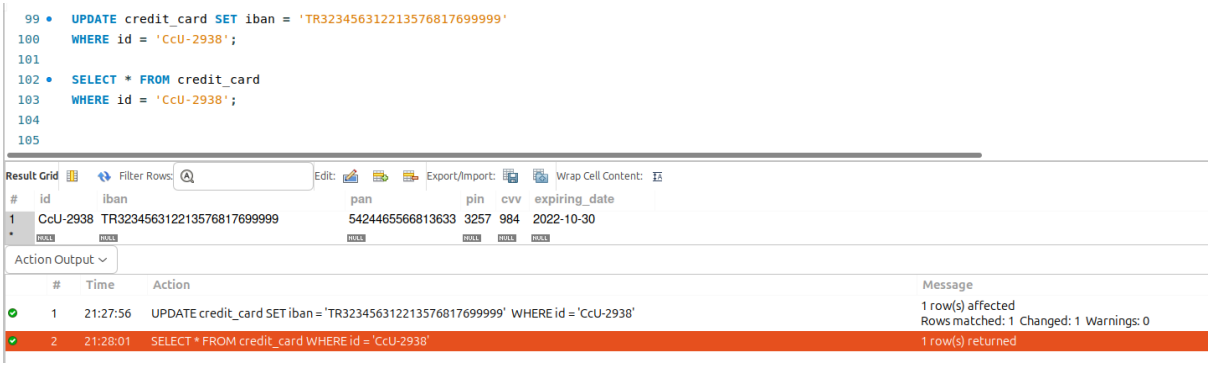



Ya tenemos el esquema completo. La tabla de 'transaction' es la de hechos mientras que 'company' y 'credit_card' representan dimensiones.

---

**Exercici 5**

**El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.**




Con update/set puede actualizar la columna de interés y con where, el registro en cuestión.

**Exercici 6**
**En la taula "transaction" ingressa una nova transacció amb la següent informació:**

```
Id

108B1D1D-5B23-A76C-55EF-C568E49A99DD

credit_card_id

CcU-9999

company_id

b-9999

user_id

9999

lat

829.999

longitude

-117.999

amount

111.11

declined

0
```

```
141 •  INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)
142    VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD','CcU-9999', 'b-9999', '9999', '829.999', '-117.999', current_timestamp(), '111.11', '0');
143
144 •  SELECT * FROM `transaction`WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| # | id | credit_card_id | company_id | user_id | lat | longitude | timestamp | amount | declined |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 108B1D1D-5B23-A76C-55EF-C568E49A99DD | CcU-9999 | b-9999 | 9999 | 829.999 | -117.999 | 2026-04-21 10:08:50 | 111.11 | 0 |
| * | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL |

transaction 28

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 10:08:50 | INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('108B1D... | 1 row(s) affected |
| 2 | 10:08:57 | SELECT * FROM `transaction`WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD' LIMIT 0, 300 | 1 row(s) returned |

Con INSERT INTO añado la nueva transacción con los datos facilitados. En la columna timestamp añadí la hora actual con CURRENT_TIMESTAMP(), es decir, la hora en la que inserté los valores. Como ya había creado esta empresa en un ejercicio anterior en la tabla ‘credit_card’, no me dio error.

**Exercici 7**
**Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.**

```
116
117 • ALTER TABLE credit_card
118       DROP COLUMN pan;
119
```

Action Output

| # | Time | Action | | Message |
|---|---|---|---|---|
| 1 | 19:03:01 | ALTER TABLE credit_card | DROP COLUMN pan | 0 row(s) affected<br>Records: 0 Duplicates: 0 Warning |

Con ALTER TABLE nombre_tabla y DROP COLUMN nombre_columna, elimino la columna pan.

---

**Exercici 8**
**Descarrega els arxius CSV que trobaràs a l'apartat de recursos:**

**american_users.csv**
**european_users.csv**
**companies.csv**
**credit_cards.csv**
**transactions.csv**
**Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:**

Primero creo la base de datos:

```
125         -- Exercici 8
126     -- creo la base de dades
127 •   CREATE DATABASE mi_negocio;
128 •   USE mi_negocio;
129
```

Action Output

| # | Time | Action |
|---|---|---|
| 1 | 22:14:37 | CREATE DATABASE mi_negocio |
| 2 | 22:14:43 | USE mi_negocio |

A continuación, para tener una idea de las tablas y poder hacer la creación y configuración, observé las primeras filas de cada una de las tablas en el terminal (aquí muestro un ejemplo de la tabla european_users) :

```
> head -n5 'N1.Ex.8__ european_users.csv'
id,name,surname,phone,email,birth_date,country,city,postal_code,address
151,Meghan,Hayden,0800 746 6747,arcu.vel@hotmail.ca,"Jul 2, 1980",United Kingdom,London,EC1A 1BB,Ap #432-4493 Aliquet Rd.
152,Hakeem,Alford,(0111) 367 0184,adipiscing.ligula@google.edu,"Sep 30, 1979",United Kingdom,Birmingham,B1 1AA,551-8930 Lobortis Street
153,Keegan,Pugh,(016977) 3851,sodales.nisi@aol.org,"Jul 27, 1994",United Kingdom,London,EC1A 1BB,Ap #312-5898 Consectetuer St.
154,Cooper,Bullock,(021) 2521 6627,et@outlook.net,"Nov 2, 1986",United Kingdom,Manchester,M1 1AE,872-1866 Pede Rd.
```

Luego fui creando cada tabla, dejando para el final la tabla de 'transactions', pues es la de hechos. Cuando creo las tablas ya configuro la PK, pero las FK las establezco más tarde.

```sql
CREATE TABLE american_users(
    id INT PRIMARY KEY,
    name VARCHAR (25),
    surname VARCHAR (50),
    phone VARCHAR (25),
    email VARCHAR(50),
    birth_date VARCHAR(22),
    country VARCHAR (30),
    city VARCHAR(30),
    postal_code VARCHAR(25),
    address VARCHAR (50)
    );
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 22:25:54 | CREATE TABLE american_users( id INT PRIMARY KEY, name VARCHAR (25), surname VARCHAR (50), phone VARCHAR (25), ema... | 0 row(s) affected |

```sql
CREATE TABLE european_users(
id INT PRIMARY KEY,
name VARCHAR (25),
surname VARCHAR (50),
phone VARCHAR (25),
email VARCHAR(50),
birth_date VARCHAR(22),
country VARCHAR (30),
city VARCHAR(30),
postal_code VARCHAR(25),
address VARCHAR (50)
);
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 22:27:41 | CREATE TABLE european_users( id INT PRIMARY KEY, name VARCHAR (25), surname VARCHAR (50), phone VARCHAR (25), email VARCH... | 0 row(s) affected |

```sql
CREATE TABLE companies (
    company_id VARCHAR (8) NOT NULL,
    company_name VARCHAR (40),
    phone VARCHAR(15),
    email VARCHAR(70),
    country VARCHAR(30),
    website VARCHAR(50),
    PRIMARY KEY (company_id)
    );
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 10:15:18 | CREATE TABLE companies ( company_id VARCHAR (8) NOT NULL, company_name VARCHAR (40), phone VARCHAR(15), e... | 0 row(s) affected |

```sql
CREATE TABLE credit_cards (
    id VARCHAR(10) PRIMARY KEY,
    user_id INT,
    iban VARCHAR(50),
    pan VARCHAR(20),
    pin SMALLINT,
    cvv SMALLINT,
    track1 VARCHAR(100),
    track2 VARCHAR(100),
    expiring_date VARCHAR(10)
    );
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 22:31:06 | CREATE TABLE credit_cards ( id VARCHAR(10) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin SMALLI... | 0 row(s) affected |

```sql
227 • CREATE TABLE transactions (
228       id VARCHAR(255) NOT NULL,
229       card_id VARCHAR(10),
230       business_id VARCHAR(8),
231       timestamp TIMESTAMP,
232       amount DECIMAL (10,2),
233       declined TINYINT,
234       product_ids VARCHAR (50),
235       user_id INT,
236       lat FLOAT(24),
237       longitude FLOAT(24),
238       PRIMARY KEY (id)
239       );
240
241   -- Carrego les dades i comprovo que s'han afegit bé
242 • SELECT * FROM transactions;
243
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 22:32:00 | CREATE TABLE transactions ( id VARCHAR(255) NOT NULL, card_id VARCHAR(10), business_id VARCHAR(8), timestamp TIM... | 0 row(s) affected |

Una vez tengo las tablas creadas, cargo los datos desde el mysql de mi terminal porque me estaba dando problemas para configurar infile=1 en workbench.

Primero accedo a mysql.

```
> mysql --local-infile=1 -u root -p mi_negocio
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 85
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Luego cargo los datos csv con el siguiente comando:

LOAD DATA LOCAL INFILE 'ruta_archivo'
INTO TABLE tabla_destino
FIELDS TERMINATED BY ',' (separador de campos)
ENCLOSED BY '"' (campos entre comillas)
LINES TERMINATED BY '\n' (salto de línea)
IGNORE 1 ROWS; (ignora la primera fila que corresponden al título campo)

En la tabla transactions, el separador es ; por lo que hay que tener en cuenta este aspecto. Las demás tablas se cargan igual, simplemente modificando la ruta y la tabla de destino.

```
    -> LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1-Ex.8__ american_users.csv'ERROR 1064 (42000): You have an error in your SQL syntax; check the manual t
hat corresponds to your MySQL server version for the right syntax to use near 'ignora la primera fila que corresponden al titulo campo)
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1-Ex.8__ american_users.csv' INTO TABLE american_users FIELDS TERMINATED BY ','  ENCLOSED BY '"'  LINES
 TERMINATED BY '\n'  IGNORE 1 ROWS;
Query OK, 1010 rows affected (0,11 sec)OSED BY '"'  LINES TERMINATED BY '\n'  IGNORE 1 ROWS;
Records: 1010  Deleted: 0  Skipped: 0  Warnings: 0
```

```
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1.Ex.8__ european_users.csv' INTO TABLE european_users FIELDS TERMINATED BY ','  ENCLOSED BY '"'  LINES
 TERMINATED BY '\n'  IGNORE 1 ROWS;
Query OK, 3990 rows affected (0,42 sec)
Records: 3990  Deleted: 0  Skipped: 0  Warnings: 0
```

```
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1.Ex.8__ companies.csv' INTO TABLE companies FIELDS TERMINATED BY ','  ENCLOSED BY '"'  LINES TERMINATE
D BY '\n'  IGNORE 1 ROWS;
Query OK, 100 rows affected (0,02 sec)
Records: 100  Deleted: 0  Skipped: 0  Warnings: 0
```

```
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1.Ex.8__ credit_cards.csv' INTO TABLE credit_cards FIELDS TERMINATED BY ','  ENCLOSED BY '"'  LINES TER
MINATED BY '\n'  IGNORE 1 ROWS;
Query OK, 5000 rows affected (0,43 sec)
Records: 5000  Deleted: 0  Skipped: 0  Warnings: 0
```

```
mysql>  LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N1.Ex.8__ transactions.csv' INTO TABLE transactions FIELDS TERMINATED BY ';'  ENCLOSED BY '"'  LINES TER
MINATED BY '\n'  IGNORE 1 ROWS;
Query OK, 100000 rows affected (1,64 sec)
Records: 100000  Deleted: 0  Skipped: 0  Warnings: 0
```

Después de confirmar que se cargaron todos los datos correctamente, decidí crear la tabla ‘users’ para unir las tablas ‘american_users’ y ‘european_users’, pues ambas tenían los mismos campos y me resultaba más fácil hacer las relaciones.

No obstante, con el objetivo de no perder ninguna información, creé la columna ‘continent’ en ambas tablas para colocar el continente al cual pertenecen. Como la actualización es para todos los registros, hice UPDATE/ SET sin WHERE y añadí el LIMIT por tema de permisos.

```
177 •  ALTER TABLE european_users
178        ADD continent CHAR (10);
179
180 •  UPDATE european_users
181        SET continent = 'europe'
182        LIMIT 5000;
183
184 •  SELECT * FROM european_users;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

| # | id | name | surname | phone | email | birth_date | country | city | postal_code | address | continent |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 151 | Meghan | Hayden | 0800 746 6747 | arcu.vel@hotmail.ca | Jul 2, 1980 | United Kingdom | London | EC1A 1BB | Ap #432-4493 Aliquet Rd. | europe |
| 2 | 152 | Hakeem | Alford | (0111) 367 0184 | adipiscing.ligula@google.edu | Sep 30, 1979 | United Kingdom | Birmingham | B1 1AA | 551-8930 Lobortis Street | europe |
| 3 | 153 | Keegan | Pugh | (016977) 3851 | sodales.nisi@aol.org | Jul 27, 1994 | United Kingdom | London | EC1A 1BB | Ap #312-5898 Consectetuer St. | europe |
| 4 | 154 | Cooper | Bullock | (021) 2521 6627 | et@outlook.net | Nov 2, 1986 | United Kingdom | Manchester | M1 1AE | 872-1866 Pede Rd. | europe |
| 5 | 155 | Joshua | Russell | 055 4409 5286 | justo.nec.ante@outlook.edu | Jan 23, 1984 | United Kingdom | Manchester | M1 1AE | Ap #285-4727 Auctor. Av. | europe |
| 6 | 156 | Remedios | Case | 055 3114 1566 | mollis.phasellus.libero@aol.com | Oct 9, 1994 | United Kingdom | Liverpool | L1 1AA | 479-3690 Turpis Road | europe |
| 7 | 157 | Philip | Carey | 0800 640 6251 | phasellus@yahoo.net | Oct 10, 1992 | United Kingdom | Manchester | M1 1AE | 196-1103 Quisque Street | europe |
| 8 | 158 | Fatima | Dyer | 0800 1111 | adipiscing@google.org | Dec 24, 1988 | United Kingdom | Birmingham | B1 1AA | Ap #435-7194 Scelerisque, St. | europe |
| 9 | 159 | Kylynn | Acevedo | 056 5727 9602 | dignissim.lacus.aliquam@google.org | May 10, 2000 | United Kingdom | Liverpool | L1 1AA | 871-3506 Lectus. Ave | europe |

european_users 59

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 23:02:48 | UPDATE european_users SET continent = 'europe' LIMIT 5000 | 3990 row(s) affected<br>Rows matched: 3990 Changed: 3990 Warnings: 0 |
| 2 | 23:03:03 | SELECT * FROM european_users | 3990 row(s) returned |

```
186 •  ALTER TABLE american_users
187        ADD continent CHAR (10);
188
189 •  UPDATE american_users
190        SET continent = 'america'
191        LIMIT 5000;
192
193 •  SELECT * FROM american_users;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

| # | id | name | surname | phone | email | birth_date | country | city | postal_code | address | continen |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | Zeus | Gamble | 1-282-581-0551 | interdum.enim@protonmail.edu | Nov 17, 1985 | United States | New York | 10001 | 348-7818 Sagittis St. | america |
| 2 | 2 | Garrett | Mcconnell | (718) 257-2412 | integer.vitae.nibh@protonmail.org | Aug 23, 1992 | United States | Philadelphia | 19101 | 903 Sit Ave | america |
| 3 | 3 | Ciaran | Harrison | (522) 598-1365 | interdum.feugiat@aol.org | Apr 29, 1998 | United States | Houston | 77001 | 736-2063 Tellus St. | america |
| 4 | 4 | Howard | Stafford | 1-411-740-3269 | ornare.egestas@icloud.edu | Feb 18, 1989 | United States | Phoenix | 85001 | Ap #545-2244 Erat. Rd. | america |
| 5 | 5 | Hayfa | Pierce | 1-554-541-2077 | et.malesuada.fames@hotmail.org | Sep 26, 1998 | United States | Philadelphia | 19101 | 341-2821 Ultrices Av. | america |
| 6 | 6 | Joel | Tyson | (718) 288-8020 | gravida.nunc.sed@yahoo.ca | Oct 15, 1989 | United States | San Jose | 95101 | 888-2799 Amet Street | america |
| 7 | 7 | Rafael | Jimenez | (817) 689-0478 | eget@outlook.ca | Dec 4, 1981 | United States | Chicago | 60601 | 8627 Malesuada Rd. | america |
| 8 | 8 | Nissim | Franks | (692) 157-3469 | egestas.aliquam.fringilla@google.ca | Aug 1, 1993 | United States | New York | 10001 | Ap #251-7144 Integer St. | america |
| 9 | 9 | Mannix | Mcclain | (590) 883-2184 | aliquam.nisl@outlook.com | Jan 24, 1987 | United States | San Antonio | 78201 | 647-3080 Lacus. St. | america |

american_users 61

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 23:05:12 | ALTER TABLE american_users ADD continent CHAR (10) | 0 row(s) affected<br>Records: 0 Duplicates: 0 Warnings: 0 |
| 2 | 23:05:19 | UPDATE american_users SET continent = 'america' LIMIT 5000 | 1010 row(s) affected<br>Rows matched: 1010 Changed: 1010 Warnings: 0 |
| 3 | 23:05:25 | SELECT * FROM european_users | 3990 row(s) returned |
| 4 | 23:05:40 | SELECT * FROM american_users | 1010 row(s) returned |

```
197 •  CREATE TABLE users AS
198        SELECT * FROM american_users
199        UNION
200        SELECT * FROM european_users;
201
202 •  ALTER TABLE users
203    ADD PRIMARY KEY (id);
204
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 23:08:20 | CREATE TABLE users AS SELECT * FROM american_users UNION SELECT * FROM european_users | 5000 row(s) affected<br>Records: 5000 Duplicates: 0 Warnings: 0 |
| 2 | 23:08:43 | ALTER TABLE users ADD PRIMARY KEY (id) | 0 row(s) affected<br>Records: 0 Duplicates: 0 Warnings: 0 |

Una vez creado el mismo campo en ambas tablas y rellenado con el continente, hice la unión en la tabla users y a continuación especifiqué la PK.

Finalmente, comprobé que la tabla final estaba bien y borré las tablas originales.

```
205 •  SELECT * FROM users;
206
207    -- BORRO les taules american_users i european_users
208 •  DROP TABLE american_users;
209 •  DROP TABLE european_users;
210
```

Action Output

| # | Time | Action |
|---|---|---|
| 1 | 23:10:53 | SELECT * FROM users |
| 2 | 23:11:03 | DROP TABLE american_users |
| 3 | 23:11:05 | DROP TABLE european_users |

- Creación de las relaciones:

1. transactions - credit_cards

```
280 • ALTER TABLE transactions
281   ADD CONSTRAINT fk_transactions_credit_cards
282   FOREIGN KEY (card_id)
283   REFERENCES credit_cards (id);
284
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 10:31:05 | ALTER TABLE transactions ADD CONSTRAINT fk_transactions_credit_cards FOREIGN KEY (card_id) REFERENCES credit_cards (id) | 99999 row(s) affected Records: 99999 Duplicates: 0 Warnings: 0 |

2. transactions - users

```
286   -- transactions-users
287 • ALTER TABLE transactions
288   ADD CONSTRAINT fk_transactions_users
289   FOREIGN KEY (user_id)
290   REFERENCES users (id);
291
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 10:34:52 | ALTER TABLE transactions ADD CONSTRAINT fk_transactions_users FOREIGN KEY (user_id) REFERENCES users (id) | 99999 row(s) affected Records: 99999 Duplicates: 0 Warnings: 0 |

3. transactions - companies

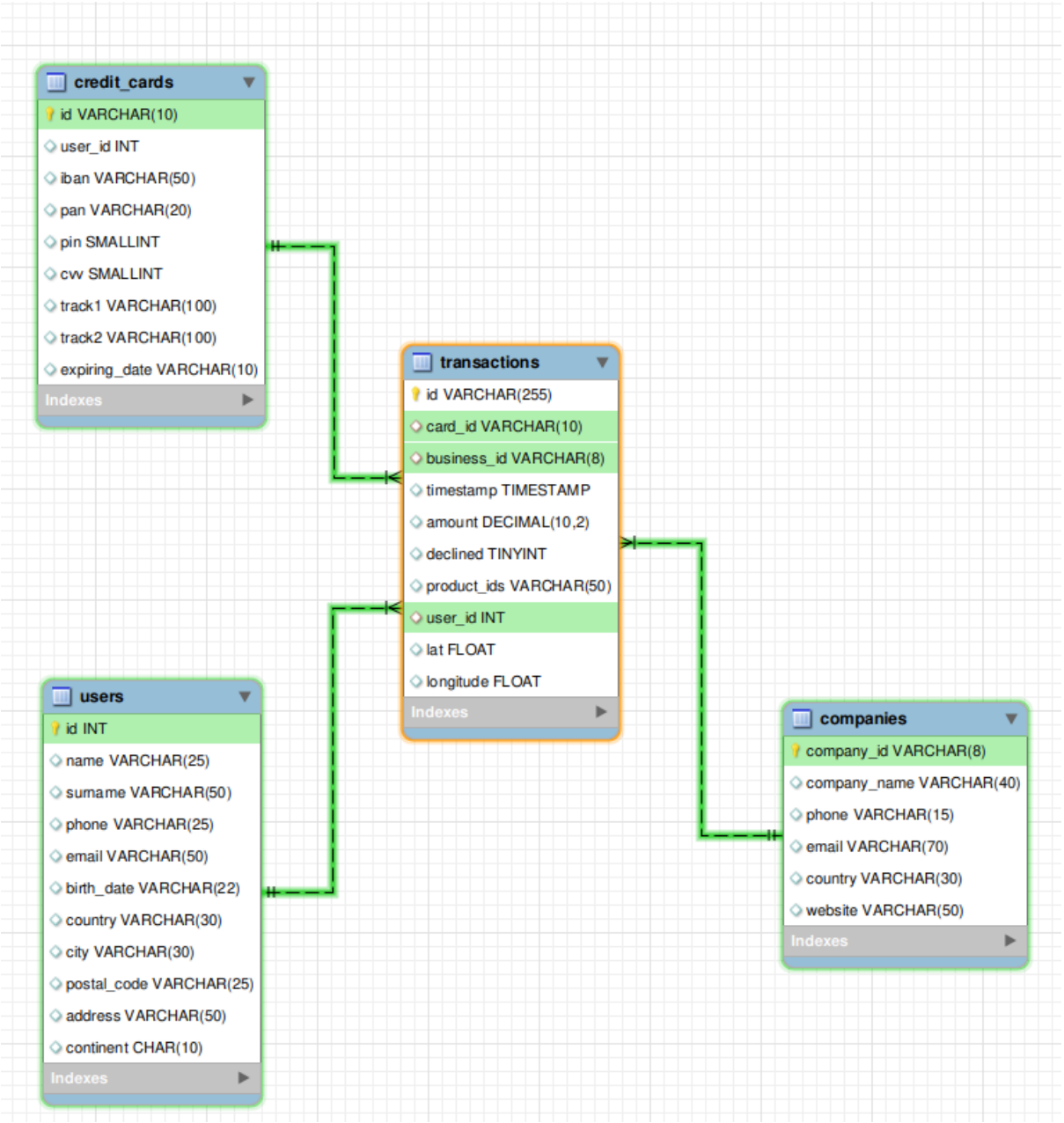
```
293 • ALTER TABLE transactions
294   ADD CONSTRAINT fk_transactions_companies
295   FOREIGN KEY (business_id)
296   REFERENCES companies (company_id);
297
298
299   -- Exercici 9
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 10:37:30 | ALTER TABLE transactions ADD CONSTRAINT fk_transactions_companies FOREIGN KEY (business_id) REFERENCES companies (com... | 99999 row(s) affected Records: 99999 Duplicates: 0 Warnings: 0 |

A continuación observamos el esquema en estrella que hemos generado, con la tabla ‘transactions’ como tabla de hechos y las tablas ‘credit-cards’, ‘users’ y ‘companies’ como dimensiones.

La relación de estas últimas con la tabla ‘transactions’ es de N a 1.


credit_cards
id VARCHAR(10)
user_id INT
iban VARCHAR(50)
pan VARCHAR(20)
pin SMALLINT
cvv SMALLINT
track1 VARCHAR(100)
track2 VARCHAR(100)
expiring_date VARCHAR(10)
Indexes
transactions
id VARCHAR(255)
card_id VARCHAR(10)
business_id VARCHAR(8)
timestamp TIMESTAMP
amount DECIMAL(10,2)
declined TINYINT
product_ids VARCHAR(50)
user_id INT
lat FLOAT
longitude FLOAT
Indexes
users
id INT
name VARCHAR(25)
surname VARCHAR(50)
phone VARCHAR(25)
email VARCHAR(50)
birth_date VARCHAR(22)
country VARCHAR(30)
city VARCHAR(30)
postal_code VARCHAR(25)
address VARCHAR(50)
continent CHAR(10)
Indexes
companies
company_id VARCHAR(8)
company_name VARCHAR(40)
phone VARCHAR(15)
email VARCHAR(70)
country VARCHAR(30)
website VARCHAR(50)
Indexes

**Exercici 9**
**Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.**

```
305 •  SELECT u.id, u.`name`, n.n_trans
306    FROM users AS u,
307                   (SELECT user_id, count(*) AS n_trans
308                   FROM transactions AS t
309                   WHERE t.declined = 0
310                   GROUP BY user_id) AS n
311    WHERE u.id = n.user_id
312    HAVING n.n_trans > 80
313    ORDER BY n.n_trans DESC;
314
315    -- Exercici 10
316    -- Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almei
```

Result Grid  Filter Rows:  Export:  Wrap Cell Content:

| # | id | name | n_trans |
|---|---|---|---|
| 1 | 185 | Molly | 107 |
| 2 | 289 | Dxwgi | 91 |
| 3 | 318 | Bnyr | 86 |

Result 123

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 19:23:23 | SELECT u.id, u.`name`, n.n_trans FROM users AS u, (SELECT user_id, count(*) AS n_trans ... | 3 row(s) returned |

Hice una subquery dentro del FROM. En la query de dentro agrupo por usuario y hago el contaje, de manera que obtengo el número de transacciones aceptadas de cada usuario. En la query de fuera, obtengo el id del usuario, el nombre y obtengo el número de transacciones al llamar a la query de dentro con el alias. Uno las dos tablas mediante el WHERE y finalmente hago el HAVING para sólo considerar aquellos usuarios con más de 80 transacciones. Para acabar, ordeno de manera descendente según el número de transacciones.

---

**Exercici 10**
**Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.**

Hago una join, con WHERE selecciono la empresa en cuestión y las transacciones aceptadas, luego agrupo por iban y con la función de agregación AVG calculo la media de amount. Finalmente, ordeno por la media de manera descendente.

```sql
SELECT cc.iban, ROUND (avg(t.amount),2) AS media
FROM transactions AS t
JOIN companies AS c
ON t.business_id = c.company_id
JOIN credit_cards AS cc
ON cc.id = t. card_id
WHERE company_name = 'Donec Ltd' AND
t.declined = 0
GROUP BY cc.iban
ORDER BY avg(amount) DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

| # | iban | media |
|---|---|---|
| 1 | XX3830178139196201993663 52 | 680.69 |
| 2 | XX6377063573975703949739 13 | 680.01 |
| 3 | XX9713939714652922023122 59 | 645.46 |
| 4 | XX1718471169288923759693 07 | 628.89 |
| 5 | XX2254246388185424062235 75 | 608.68 |
| 6 | XX7488907290571957117660 71 | 607.29 |
| 7 | TN9614563570667381893122 | 605.41 |
| 8 | XX4819080340373642425911 85 | 605.36 |
| 9 | XX1946755197392563357535 08 | 597.19 |
| 10 | XX2159627660619671954934 37 | 594.26 |
| 11 | XX4493223208268907210014 43 | 591.61 |
| 12 | XX5351854927357042294742 37 | 570.09 |
| 13 | CH9552373968796160224 | 566.38 |
| 14 | XX3476053771256378803031 31 | 561.80 |
| 15 | XX6884714466979219128603 04 | 543.42 |

Result 64

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 12:09:33 | SELECT cc.iban, ROUND (avg(t.amount),2) AS media FROM transactions AS t JOIN companies AS c ON t.b... | 300 row(s) returned |

## 2. Nivell 2

**Exercici 1**
**Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.**

```
SELECT date(t.`timestamp`) AS fecha, sum(amount) total_ventas
FROM transactions AS t
JOIN companies AS c
ON t.business_id = c.company_id
WHERE declined = 0
GROUP BY date(t.`timestamp`)
ORDER BY total_ventas DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

| # | fecha | total_ventas |
|---|---|---|
| 1 | 2022-12-13 | 14337.44 |
| 2 | 2019-11-18 | 13591.32 |
| 3 | 2023-02-20 | 13332.59 |
| 4 | 2017-12-20 | 13318.43 |
| 5 | 2019-03-18 | 12680.95 |

Result 74

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 12:20:14 | SELECT date(t.`timestamp`) AS fecha, sum(amount) total_ventas FROM transactions AS t JOIN comp... | 5 row(s) returned |

Calculé los 5 días que han generado más ingresos. Hice una join, agrupé por fecha con la función DATE y calculé el total de amount por día. Consideré sólo las transacciones aceptadas. Finalmente ordené y mostré solo los 5 primeros registros.

---

**Exercici 2**
**Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.**

Hago el JOIN entre 'companies' y 'transactions' y luego establezco los siguientes filtros:

- Transacciones aceptadas
- Solo considero las fechas que proporciona el enunciado
- Transacciones con valores entre 350 y 400

Finalmente, ordeno en orden descendente el amount.

```sql
SELECT c.company_name, c.phone, c.country, date(t.`timestamp`) AS fecha, t.amount AS cantidad
FROM companies AS c
JOIN transactions AS t
ON c.company_id = t.business_id
WHERE declined = 0
AND (date(t.`timestamp`) IN ('2015-04-29', '2018-7-20','2024-03-13'))
AND (t.amount BETWEEN 350 AND 400)
ORDER BY t.amount DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | company_name | phone | country | fecha | cantidad |
|---|---|---|---|---|---|
| 1 | Aliquam PC | 01 45 73 52 16 | Germany | 2024-03-13 | 399.84 |
| 2 | Auctor Mauris Vel LLP | 08 09 28 74 14 | United States | 2018-07-20 | 399.51 |
| 3 | At Pede Corp. | 06 14 48 33 15 | Italy | 2015-04-29 | 390.69 |
| 4 | Aliquam PC | 01 45 73 52 16 | Germany | 2024-03-13 | 388.29 |
| 5 | Orci Adipiscing Limited | 03 18 00 77 81 | United Kingdom | 2018-07-20 | 373.71 |
| 6 | Fringilla LLC | 08 29 15 93 57 | New Zealand | 2015-04-29 | 367.62 |
| 7 | Pede Cum Ltd | 07 62 26 48 38 | Norway | 2018-07-20 | 356.87 |
| 8 | Auctor Mauris Vel LLP | 08 09 28 74 14 | United States | 2024-03-13 | 353.75 |

Result 86

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 12:45:46 | SELECT c.company_name, c.phone, c.country, date(t.`timestamp`) AS fecha, t.amount AS cantidad FRO... | 8 row(s) returned |

**Exercici 3**

**Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen més de 400 transaccions o menys.**

Primero hice una JOIN, agrupé por empresa y con count(*) calculé el total de transacciones de cada empresa. A continuación, usé CASE para establecer una columna nueva indicando si el número de transacciones está por encima o por debajo de 400 transacciones. Finalmente, utilicé ROW_NUMBER grupando por “por encima” o “por debajo” y ordene por el número de transacciones. Así me queda una lista de las empresas de cada grupo.

```sql
SELECT *, ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY num_transactions ASC) AS lista
FROM
    (SELECT *, CASE
            WHEN num_transactions < 400 THEN 'Por debajo'
            ELSE 'Por encima'
            END AS categoria
    FROM

            (SELECT
                c.company_id,
                c.company_name,
                count(*) AS num_transactions
            FROM transactions AS t
            JOIN companies AS c
            ON t.business_id = c.company_id
            WHERE declined = 0
            GROUP BY c.company_id, c.company_name) AS nt) AS nu;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | company_ic | company_name | num_transaction | categoria | lista |
|---|---|---|---|---|---|
| 1 | b-2362 | Lorem Eu Incorporated | 378 | Por debajo | 1 |
| 2 | b-2286 | Fringilla LLC | 396 | Por debajo | 2 |
| 3 | b-2470 | Nec Luctus LLC | 399 | Por debajo | 3 |
| 4 | b-2578 | Dui Quis Institute | 400 | Por encima | 1 |
| 5 | b-2226 | Magna A Neque Industries | 406 | Por encima | 2 |
| 6 | b-2446 | Risus Associates | 407 | Por encima | 3 |
| 7 | b-2526 | Amet Luctus Vulputate Foundation | 410 | Por encima | 4 |
| 8 | b-2534 | Mattis Foundation | 412 | Por encima | 5 |
| 9 | b-2590 | Euismod Mauris Institute | 417 | Por encima | 6 |

Result 7

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 10:15:45 | SELECT *, ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY num_transactions ASC) AS lista FROM (SELECT *, CASE ... | 100 row(s) returned |

## Exercici 4

**Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.**

```sql
DELETE FROM transactions
WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 10:19:44 | DELETE FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' | 1 row(s) affected |

Con DELETE y WHERE elimino las transacción dada.

**Exercici 5**

**La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.**

```
439 •  CREATE OR REPLACE VIEW VistaMarketing AS
440    SELECT c.company_name,
441           c.phone, c.country,
442           ROUND(AVG(amount),2) AS media
443    FROM companies AS c
444    JOIN transactions AS t
445    ON c.company_id = t.business_id
446    WHERE t.declined = 0
447    GROUP BY c.company_id, c.company_name, c.phone, c.country
448    ORDER BY ROUND(AVG(amount),2) DESC;
449
450 •  SELECT *
451    FROM VistaMarketing
452    ORDER BY media DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

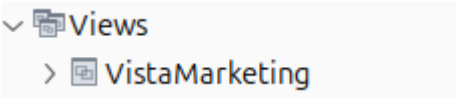
| # | company_name | phone | country | media |
|---|---|---|---|---|
| 1 | Ac Fermentum Incorp... | 06 85 56 52 33 | Germany | 284.91 |
| 2 | Pretium Neque Corp. | 07 77 48 55 28 | Australia | 275.58 |
| 3 | Urna Convallis Associ... | 06 01 24 77 04 | United States | 273.57 |
| 4 | At Associates | 09 56 61 10 65 | New Zealand | 272.74 |
| 5 | Metus Vitae Associates | 08 25 44 40 66 | Australia | 270.05 |
| 6 | Aliquet Diam Limited | 02 76 61 47 46 | United States | 269.29 |
| 7 | Nec Luctus LLC | 02 14 71 75 73 | Norway | 268.60 |
| 8 | Neque Tellus Incorpor... | 04 43 18 34 19 | Ireland | 267.56 |
| 9 | Cras Consulting | 07 50 10 85 63 | Belgium | 267.38 |
| 10 | Sed LLC | 01 63 16 26 52 | Belgium | 266.61 |

VistaMarketing 87

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 12:56:37 | CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.company_name, c.phone, c.country, ... | 0 row(s) affected |
| 2 | 12:56:42 | SELECT * FROM VistaMarketing ORDER BY media DESC LIMIT 0, 300 | 100 row(s) returned |

Creo la vista con CREATE VIEW AS. En la query hago una JOIN, agrupo por empresa, calculo la media de amount y muestro los campos que se piden. Para consultar la visa ordené ya que al abrir la vista puede que no sirva el ORDER BY que contiene la vista.



## 3. Nivell 3

**Nivell 3**
**Exercici 1**
**Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:**

Los pasos que seguí se describen a continuación:

- Agrupo por tarjeta de credito, ordeno el resultado por fecha empezando por las transacciones más recientes y creo un listado de todos los registros de cada targeta
- A continuación, con el filtro WHERE selecciono de la lista los 3 primeros registros de cada tarjeta de crédito, es decir, me quedo con las 3 transacciones más recientes de cada una
- Luego hice un sumatorio acumulativo con OVER de manera que sumé el 'declined' de las 3 transacciones más recientes de cada tarjeta y coloque el resultado en una nueva columna llamada 'transacciones_declinadas'. El sumatorio se acumulava en la fila tercera de cada tarjeta, es decir, es la fila que nos interesa porque nos muestra las cantidad de transacciones declinadas
- Finalmente, establecí un condicional con CASE en función del valor de la fila 3; si este valor es = 3 → tarjeta inactiva / else → tarjeta activa. El estado de la tarjeta se refleja en otra columna llamada 'estado_targeta'
- A continuación, cree la tabla nueva (card_status)
- Para obtener el resultado a la pregunta, tenemos que hacer un SELECT (*) de esta tabla con el estado_targeta='activa';

```
468     (SELECT card_id,
469             CASE
470                 WHEN transacciones_declinadas = '3' THEN 'inactiva'
471                 ELSE 'activa'
472             END AS estado_targeta
473     FROM
474                 (SELECT *, SUM(declined) OVER (PARTITION BY card_id ORDER BY lista) AS transacciones_declinada
475                 FROM
476                         (SELECT card_id, declined, `timestamp`,
477                         ROW_NUMBER() OVER(PARTITION BY card_id ORDER BY `timestamp` DESC) AS lista
478                         FROM transactions) AS ls
479                 WHERE lista BETWEEN '1' AND '3') AS suma_acumulativa
480     WHERE lista = '3') tabla_final;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | card_id | estado_targeta |
|---|---|---|
| 1 | CcS-4857 | activa |
| 2 | CcS-4858 | activa |
| 3 | CcS-4859 | activa |
| 4 | CcS-4860 | activa |
| 5 | CcS-4861 | activa |
| 6 | CcS-4862 | activa |
| 7 | CcS-4863 | activa |
| 8 | CcS-4864 | activa |
| 9 | CcS-4865 | activa |
| 10 | CcS-4866 | activa |

Result 92

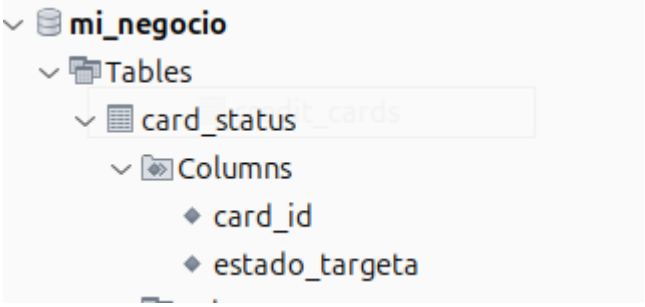
Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 13:09:59 | SELECT card_id, CASE WHEN transacciones_declinadas = '3' THEN 'inactiva' ... | 5000 row(s) returned |

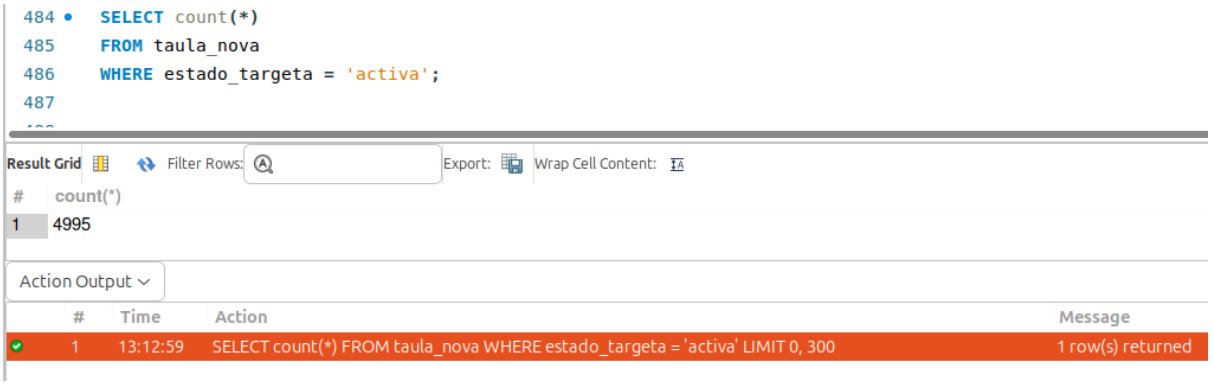
```
464 •   CREATE TABLE card_status AS
465     SELECT *
466     FROM
467
468         (SELECT card_id,
469                 CASE
470                     WHEN transacciones_declinadas = '3' THEN 'inactiva'
471                     ELSE 'activa'
472                 END AS estado_targeta
473         FROM
474                     (SELECT *, SUM(declined) OVER (PARTITION BY card_id ORDER BY lista) AS transacciones_declinadas
475                     FROM
476                             (SELECT card_id, declined, `timestamp`,
477                             ROW_NUMBER() OVER(PARTITION BY card_id ORDER BY `timestamp` DESC) AS lista
478                             FROM transactions) AS ls
479                     WHERE lista BETWEEN '1' AND '3') AS suma_acumulativa
480         WHERE lista = '3') tabla_final;
481
482
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 13:11:41 | CREATE TABLE card_status AS SELECT * FROM (SELECT card_id, CASE WHEN tra... | 5000 row(s) affected<br>Records: 5000 Duplicates: 0 Warnings: 0 |




👉 Quantes targetes estan actives?




**Exercici 2**
**Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:**

Primero voy a crear la tabla 'products'. No obstante, previamente a la creación, hago una primera observación del archivo desde mi terminal, así puedo observar los diferentes campos y tener una idea más realista de los datos que voy a importar. También me sirve para establecer el tipo de dato que mejor encaja en cada campo a la hora de la creación de la tabla (véase las dos siguientes imágenes).

Para evitar problemas a la hora de la importación, decidí que los campos serían VARCHAR (excepto en id), pues considero que es el tipo de dato que da menos problemas para dicho fin.

```
==> N3.Ex.2__ products.csv <==
id,product_name,price,colour,weight,warehouse_id
1,Direwolf Stannis,$161.11,#7c7c7c,1,WH-4
2,Tarly Stark,$9.24,#919191,2,WH-3
3,duel tourney Lannister,$171.13,#d8d8d8,1.5,WH-2
4,warden south duel,$71.89,#111111,3,WH-1
5,skywalker ewok,$171.22,#dbdbdb,3.2,WH-0
6,dooku solo,$136.60,#c4c4c4,0.8,WH--1
7,north of Casterly,$63.33,#b7b7b7,0.6,WH--2
8,Winterfell,$32.37,#383838,1.4,WH--3
9,Winterfell,$76.40,#b5b5b5,1.2,WH--4
```

```
499 • CREATE TABLE products(
500     id INT NOT NULL,
501     product_name VARCHAR (30),
502     price VARCHAR(10),
503     colour VARCHAR (12),
504     weight VARCHAR(10),
505     warehouse_id VARCHAR (9),
506     PRIMARY KEY (id));
507
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 13:56:25 | CREATE TABLE products( id INT NOT NULL, product_name VARCHAR (30), price VARCHAR(10), colour V... | 0 row(s) affected |

A continuación, me conecto a mysql, y accedo a la base de datos ‘mi_negocio’ configurando infile=1. Finalmente, cargo los datos a la tabla y compruebo que se cargaron bien (véase las tres siguientes imágenes).

```
>  mysql --local-infile=1 -u root -p mi_negocio
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 89
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

```
mysql> LOAD DATA LOCAL INFILE '/home/arnau/IT_ACADEMY/especialitzacio/sprint_2/N3.Ex.2__ products.csv'
    -> INTO TABLE products
    -> FIELDS TERMINATED BY ','
    -> ENCLOSED BY '"'
    -> LINES TERMINATED BY '\n'
    -> IGNORE 1 ROWS;
Query OK, 100 rows affected (0,02 sec)
Records: 100  Deleted: 0  Skipped: 0  Warnings: 0

mysql>
```

```
64 •  SELECT * FROM products;
65
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| # | id | product_name | price | colour | weight | warehouse_ic |
|---|---|---|---|---|---|---|
| 1 | 1 | Direwolf Stannis | $161.11 | #7c7c7c | 1 | WH-4 |
| 2 | 2 | Tarly Stark | $9.24 | #919191 | 2 | WH-3 |
| 3 | 3 | duel tourney Lannister | $171.13 | #d8d8d8 | 1.5 | WH-2 |
| 4 | 4 | warden south duel | $71.89 | #111111 | 3 | WH-1 |
| 5 | 5 | skywalker ewok | $171.22 | #dbdbdb | 3.2 | WH-0 |
| 6 | 6 | dooku solo | $136.60 | #c4c4c4 | 0.8 | WH--1 |
| 7 | 7 | north of Casterly | $63.33 | #b7b7b7 | 0.6 | WH--2 |
| 8 | 8 | Winterfell | $32.37 | #383838 | 1.4 | WH--3 |
| 9 | 9 | Winterfell | $76.40 | #b5b5b5 | 1.2 | WH--4 |
| 10 | 10 | Karstark Dorne | $119.52 | #f4f4f4 | 2.4 | WH--5 |
| 11 | 11 | Karstark Dorne | $49.70 | #141414 | 2.7 | WH--6 |

products 44

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 11:18:25 | SELECT * FROM products | 100 row(s) returned |

Como se observa en la imagen anterior, el campo 'price' aparece con el símbolo ($) en todos los registros. Como esto puede dificultar cálculos posteriores, se decidió por quitar este símbolo con UPDDATE / SET y la función REPLACE. Una vez hecha la sustitución, comprobé que efectivamente se lleva a cabo (ver las dos siguientes imágenes).

```
62
63 •  UPDATE products
64    SET price =
65    REPLACE (price, '$', '')
66    LIMIT 500;
67
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 18:27:12 | UPDATE products SET price = REPLACE (price, '$', '') LIMIT 500 | 100 row(s) affected<br>Rows matched: 100 Changed: 100 Warnings: 0 |

```
69 •  SELECT price FROM products;
70
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | price |
|---|---|
| 1 | 161.11 |
| 2 | 9.24 |
| 3 | 171.13 |
| 4 | 71.89 |
| 5 | 171.22 |
| 6 | 136.60 |
| 7 | 63.33 |
| 8 | 32.37 |
| 9 | 76.40 |
| 10 | 119.52 |
| 11 | 49.70 |

products 104

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 18:35:24 | SELECT price FROM products LIMIT 0, 300 | 100 row(s) returned |

Luego modifiqué el tipo de columna ‘price’ a DECIMAL, ya que inicialmente la clasificamos como VARCHAR para poder cargar los datos sin problemas. El tipo de dato DECIMAL recibe 2 parámetros, el primero se corresponde con el total de cifras del valor y el segundo con el número de decimales. Me decidí por un total de 6 cifras (por si acaso) y dos decimales, ya que por defecto así viene.

```
529 •  ALTER TABLE products
530    MODIFY COLUMN price DECIMAL(6,2);
531
532 •  SELECT price FROM products;
533
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | price |
|---|---|
| 1 | 161.11 |
| 2 | 9.24 |
| 3 | 171.13 |
| 4 | 71.89 |
| 5 | 171.22 |
| 6 | 136.60 |
| 7 | 63.33 |
| 8 | 32.37 |
| 9 | 76.40 |
| 10 | 119.52 |
| 11 | 49.70 |
| 12 | 181.60 |
| 13 | 139.59 |
| 14 | 147.53 |
| 15 | 194.29 |
| 16 | 180.91 |
| 17 | 91.89 |
| 18 | 148.91 |
| 19 | 60.33 |
| 20 | 91.96 |

products 99

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 14:07:54 | ALTER TABLE products MODIFY COLUMN price DECIMAL(6,2) | 100 row(s) affected<br>Records: 100 Duplicates: 0 Warnings: 0 |
| 2 | 14:08:14 | SELECT price FROM products LIMIT 0, 300 | 100 row(s) returned |

A la hora de hacer el diagrama, resulta que la relación entre la tabla ‘products’ y ‘transactions’ es n:n, por lo que hace falta una tabla intermedia ya que este tipo de relaciones es mejor evitarla.

Esta nueva tabla tendrá dos campos, transaction_id (que la vinculará con la tabla transaction) y product_id (que la vinculará con la tabla products). La PK será la combinación de ambos campos, generando una PK única para cada producto y transacción.

Respecto a las FK, configuramos cada columna como una FK, de tal manera que facilite la relación con las tablas de las cuales provienen. Así, la FK transaction_id nos permitirá relacionarnos con la tabla ‘transactions’ y la KF product_id nos permitirá relacionarnos con la tabla products.

```
534 • CREATE TABLE transactions_products (
535   transaction_id VARCHAR(255),
536   product_id INT,
537   PRIMARY KEY (transaction_id, product_id),
538   FOREIGN KEY (transaction_id) REFERENCES transactions(id),
539   FOREIGN KEY (product_id) REFERENCES products(id)
540   );
541
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 14:12:35 | CREATE TABLE transactions_products ( transaction_id VARCHAR(255), product_id INT, PRIMARY KEY (tra... | 0 row(s) affected |

Una vez creada la tabla, el siguiente paso es rellenarla. Para ello usé la función JSON_TABLE, pues me permite expandir las filas de la columna 'product_id' de la tabla 'transactions' e insertarlas en la nueva tabla (transactions-products).

Con el INSERT inserté en la columna 'transactions_id' las transacciones y en 'product_id' los productos. Con JSON_TABLE, conseguimos modificar el formato de la columna, convirtiéndolo en un array de strings de formato JSON. Para ello, usamos la función CONCAT con [ ], y la función REPLACE que nos va a sustituir la coma de entre los números por ",". Es decir, hacemos la siguiente transformación: 1,2,3 → ["1", "2", "3"] para cada uno de los registros.

A continuación, le añadimos el path, que en este caso es: ''$[*]. Estos nos sirve para generar filas de datos relacionados. Es decir, le estamos diciendo que nos haga tantas filas como valores aparecen en la fila original. En 'columns', le decimos la columna donde queremos que se añadan las filas expandidas, el tipo de dato y el path, indicando que encuentre los valores JSON y que me los devuelva expandidos en formato sql en filas individuales. También hay que añadirle un alias final.

Para evitar importar valores nulos o registros vacíos a la nueva tabla, establecemos un filtro con WHERE, de manera que evitamos los valores 'product_ids' de la tabla 'transactions' que cumplan alguna de estas condiciones.

En la siguiente imagen se observa el código explicado anteriormente.

```
545     (SELECT t.id AS transactions_id, jt.product_id
546     FROM transactions AS t,
547     JSON_TABLE (
548         CONCAT('["', REPLACE (product_ids, ',' , '","'), '"]'),
549         '$[*]' COLUMNS (
550                         product_id INT PATH "$")) AS jt
551 WHERE t.product_ids IS NOT NULL AND t.product_ids <> '');
552
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | transactions_id | product_i |
|---|---|---|
| 1 | 00043A49-2949-494B-A5DD-A5B... | 16 |
| 2 | 00043A49-2949-494B-A5DD-A5B... | 26 |
| 3 | 00043A49-2949-494B-A5DD-A5B... | 97 |
| 4 | 00043A49-2949-494B-A5DD-A5B... | 87 |
| 5 | 00045D6B-ED2E-4F2F-8186-CEE... | 30 |
| 6 | 00045D6B-ED2E-4F2F-8186-CEE... | 11 |
| 7 | 00045D6B-ED2E-4F2F-8186-CEE... | 16 |
| 8 | 00045D6B-ED2E-4F2F-8186-CEE... | 81 |
| 9 | 000481C3-1C26-4FEF-83A0-4CD... | 72 |
| 10 | 00051AA4-9CBE-4268-B070-C38... | 18 |
| 11 | 0008A312-EDFE-4A4F-BC99-E9C... | 35 |
| 12 | 0008A312-EDFE-4A4F-BC99-E9C... | 33 |

Result 103

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 17:40:41 | SELECT t.id AS transactions_id, jt.product_id FROM transactions AS t, JSON_TABLE ( CONCAT('... | 253388 row(s) returned |

*Este es el código.*

En las siguientes imágenes, primero hago la inserción de los datos y más tarde (las otras dos imágenes) compruebo que se haya realizado con éxito.

```
544 • INSERT INTO transactions_products(transaction_id, product_id)
545     (SELECT t.id AS transactions_id, jt.product_id
546     FROM transactions AS t,
547     JSON_TABLE (
548         CONCAT('["', REPLACE (product_ids, ',' , '","'), '"]'),
549         '$[*]' COLUMNS (
550                         product_id INT PATH "$")) AS jt
551 WHERE t.product_ids IS NOT NULL AND t.product_ids <> '');
552
553
```

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 17:41:41 | INSERT INTO transactions_products(transaction_id, product_id) (SELECT t.id AS transactions_id, jt.pro... | 253388 row(s) affected<br>Records: 253388 Duplicates: 0 Warnings: 0 |

```
555 • SELECT count(*) FROM transactions_products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | count(*) |
|---|---|
| 1 | 253388 |

Result 116

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 17:55:24 | SELECT count(*) FROM transactions_products LIMIT 0, 300 | 1 row(s) returned |

```
556 •  SELECT count(DISTINCT product_id)
557    FROM transactions_products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

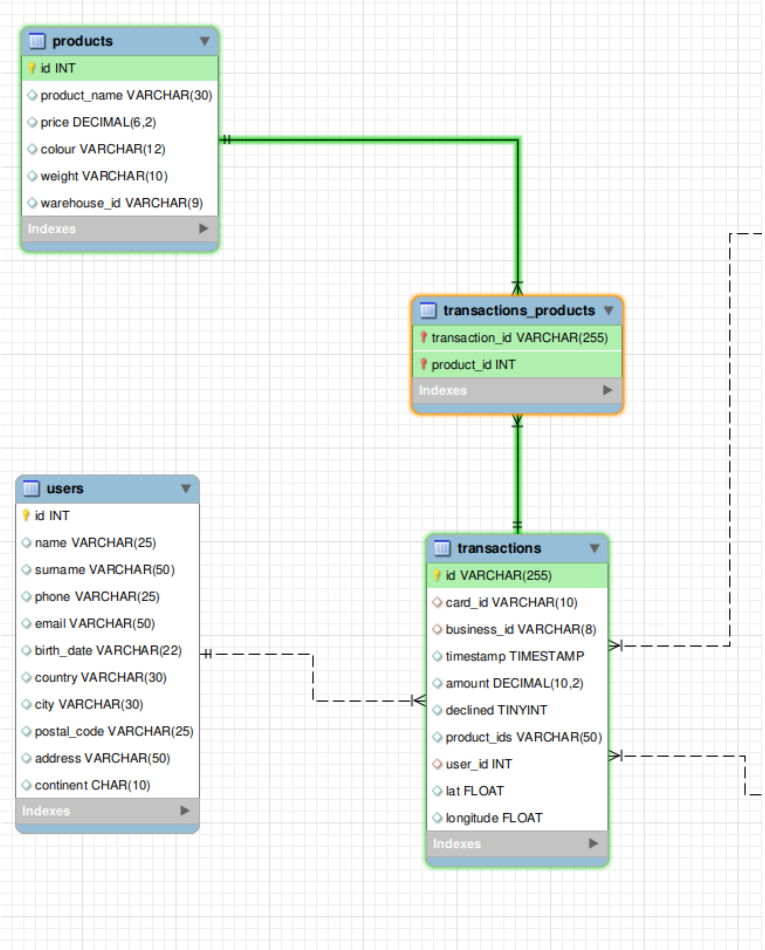
| # | count(DISTINCT product_i |
|---|---|
| 1 | 100 |

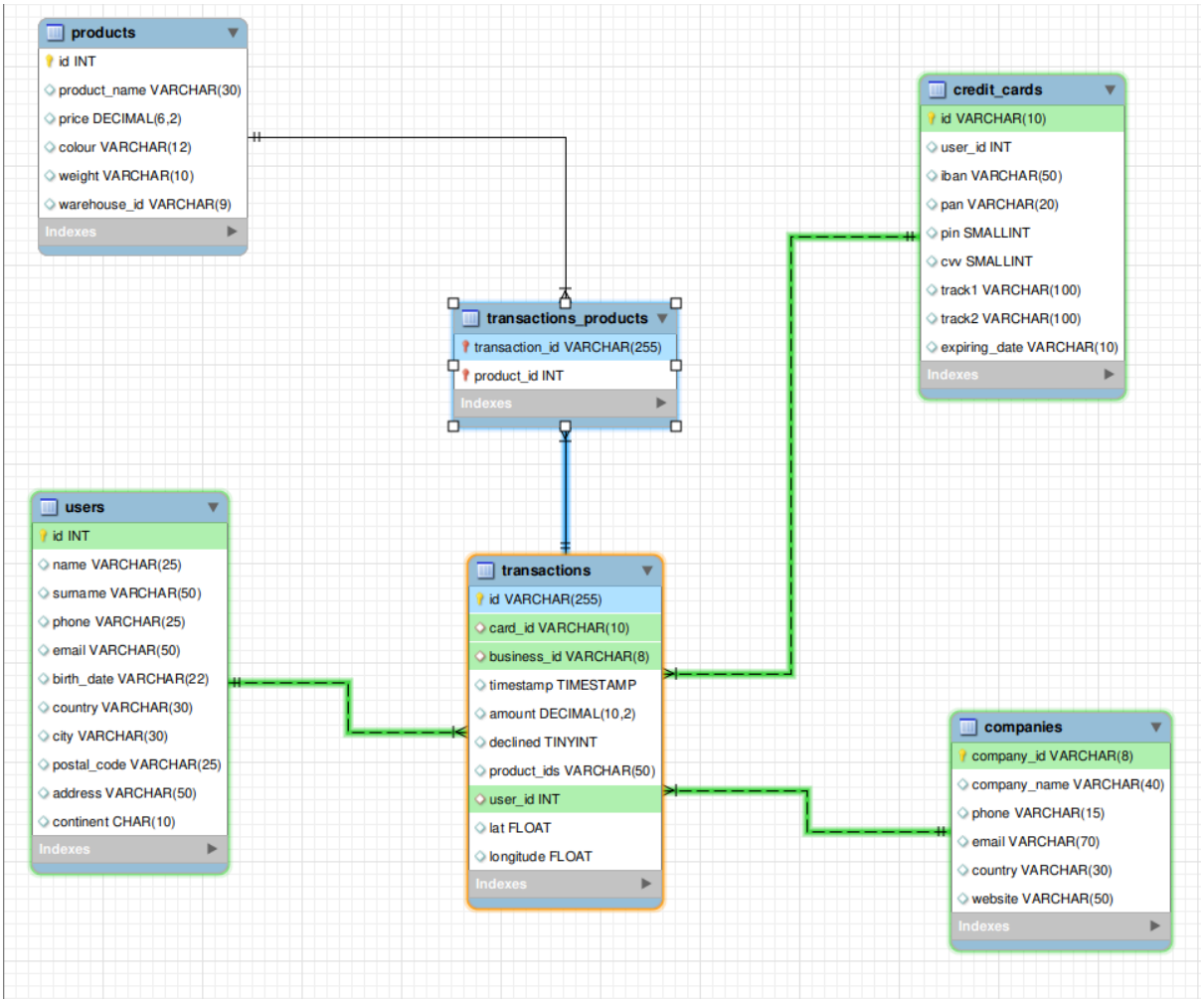
Result 117

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 17:57:27 | SELECT count(DISTINCT product_id) FROM transactions_products LIMIT 0, 300 | 1 row(s) returned |

El esquema final queda como a continuación se muestra, con una nueva tabla de hechos. Realmente son dos las tablas de hechos: 'transactions' y 'transactions_products'. Pero considero que la 'transactions_products' es la tabla de hechos principal pues quedan reflejadas exclusivamente las transacciones, mientras que la tabla transactions sería más secundária, pues aparte de las transacciones, aporta otra información adicional.




products
id INT
product_name VARCHAR(30)
price DECIMAL(6,2)
colour VARCHAR(12)
weight VARCHAR(10)
warehouse_id VARCHAR(9)
Indexes
credit_cards
id VARCHAR(10)
user_id INT
iban VARCHAR(50)
pan VARCHAR(20)
pin SMALLINT
cvv SMALLINT
track1 VARCHAR(100)
track2 VARCHAR(100)
expiring_date VARCHAR(10)
Indexes
transactions_products
transaction_id VARCHAR(255)
product_id INT
Indexes
users
id INT
name VARCHAR(25)
surname VARCHAR(50)
phone VARCHAR(25)
email VARCHAR(50)
birth_date VARCHAR(22)
country VARCHAR(30)
city VARCHAR(30)
postal_code VARCHAR(25)
address VARCHAR(50)
continent CHAR(10)
Indexes
transactions
id VARCHAR(255)
card_id VARCHAR(10)
business_id VARCHAR(8)
timestamp TIMESTAMP
amount DECIMAL(10,2)
declined TINYINT
product_ids VARCHAR(50)
user_id INT
lat FLOAT
longitude FLOAT
Indexes
companies
company_id VARCHAR(8)
company_name VARCHAR(40)
phone VARCHAR(15)
email VARCHAR(70)
country VARCHAR(30)
website VARCHAR(50)
Indexes

👉 **Necessitem conèixer el nombre de vegades que s'ha venut cada producte.**

```
560 •  SELECT p.id, p.product_name, count(*) AS num_ventas
561    FROM transactions_products tp
562    JOIN products p
563    ON tp.product_id = p.id
564    GROUP BY p.id
565    ORDER BY num_ventas DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| # | id | product_name | num_ventas |
|---|---|---|---|
| 1 | 52 | riverlands the duel | 2654 |
| 2 | 29 | Tully maester Tarly | 2635 |
| 3 | 21 | duel Direwolf | 2609 |
| 4 | 16 | the duel warden | 2608 |
| 5 | 66 | mustafar jinn | 2600 |
| 6 | 48 | rock Renly in | 2597 |
| 7 | 33 | duel warden | 2597 |
| 8 | 87 | sith Jade | 2597 |
| 9 | 23 | riverlands north | 2593 |
| 10 | 68 | Stark Karstark | 2589 |
| 11 | 88 | Stannis warden ... | 2587 |
| 12 | 28 | chewbacca must... | 2584 |
| 13 | 73 | Dorne bastard | 2584 |
| 14 | 4 | warden south duel | 2584 |
| 15 | 34 | of north | 2580 |
| 16 | 31 | Lannister | 2580 |

Result 118

Action Output

| # | Time | Action | Message |
|---|---|---|---|
| 1 | 17:58:13 | SELECT p.id, p.product_name, count(*) AS num_ventas FROM transactions_products tp JOIN products p ... | 100 row(s) returned |

Hago una JOIN entre ‘productos’ y ‘transactions_products’, agrupo por id del producto y cuento los registros obteniendo el número de ventas de cada producto. Finalmente ordeno por el número de ventas en orden descendente.